문자 수준 CNN과 DistilBERT 기반 캐스케이드 웹 공격 탐지의 속도–정확도 교환 분석

Speed–Accuracy Trade-off Analysis of Cascaded Character-Level CNN and DistilBERT for Web Attack Detection

요 약

인라인 웹 애플리케이션 방화벽(web application firewall, WAF)에서 다중 웹 공격을 탐지할 때, DistilBERT와 같은 사전학습 트랜스포머는 긴 문맥을 포착해 국소 구분자가 유사한 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI) 간 공격 종류 오탐을 줄이지만 단일 스트림 CPU 추론 지연이 커 실시간 배치에 부담이 되고, 경량 문자 수준 합성곱 신경망(character-level convolutional neural network, char-CNN)은 추론은 빠르나 짧은 수용 영역으로 인해 공격 종류 혼동을 남긴다. 이를 절충하려는 2단계 캐스케이드와 확신도 기반 넘김(deferral) 기법이 제안되어 왔으나, 전처리를 포함한 단일 스트림 종단간 지연과 낮은 실운영 공격 기저율·실사용자 정상 트래픽 분포, 그리고 다중 분할 seed에서도 속도–정확도 이득과 넘김 판단의 유효성이 유지되는지는 검증되지 않았다. 이에 본 논문은 실트래픽 기반 SR-BH 2020 데이터셋에서 사전 등록된 판정 기준에 따라 char-CNN과 DistilBERT를 결합한 딥러닝 캐스케이드, 내부 조기 종료 구조, 예측 클래스별 임계값(G2), 비용 기반 지연 유발 공격면을 단일 스레드 CPU(배치 1) 종단간 지연과 실사용자 정상 트래픽 기준으로 평가한다. 실험 결과, char-CNN 내부 분기에 보조 헤드를 결합 학습한 조기 종료 구조는 모델 압축 효과로 단독 모델과 동등한 정확도를 **3.5~3.6**배 빠르게 달성하였고, char-CNN → DistilBERT 캐스케이드는 시험 요청의 0.7~0.8%만 넘기면서 SQLi→CmdI 오탐 감소분의 **56~74%**를 흡수하며 재구현 DistilBERT 단독 대비 합산 **10.6~12.3**배(결합 학습 출구를 1차로 결합 시 **23.8~29.3**배) 가속을 얻었다. 그러나 사전 등록된 Macro-F1 손실 기준(0.30%p 이내)은 기본 seed 42(**−0.03%p**)에서만 충족하고 확인용 seed 43(**−0.32%p**)에서는 미세조정 편차로 재현되지 않았으며, 클래스별 임계값 역시 새 seed 확인에서 유의한 이득을 내지 못했고 URL 인코딩 1회 변형 시 공격 요청당 기대 지연이 **6.4**배 증가하며 1차 탐지 회피가 발생함을 확인하였다. 결론적으로 웹 공격 탐지 캐스케이드의 성패는 넘김 점수 조정이 아니라 1·2차 지연 격차와 1차 모델의 판별력 및 강건성에 달려 있으며, 캐스케이드의 성능 주장은 반드시 종단간 지연, 배포 기저율, 실사용자 정상 넘김률, 다중 seed 분산을 함께 보고할 때에만 성립한다.

ABSTRACT

In inline web application firewalls (WAFs), pre-trained transformers such as DistilBERT capture long-range context and reduce attack-type confusion between syntactically similar SQL injection (SQLi) and command injection (CmdI), but their single-stream CPU latency hinders real-time deployment, whereas lightweight character-level convolutional neural networks (char-CNNs) run fast but leave attack-type confusion due to short receptive fields. Two-stage cascades and confidence-based deferral have been proposed to bridge this gap, yet it remains unverified whether their speed–accuracy gains and deferral mechanisms hold under end-to-end single-stream latency, low deployment attack base rates with real-user benign traffic, and multiple split seeds. In this paper, we evaluate all-deep-learning cascades pairing char-CNN with fine-tuned DistilBERT, internal early exit, class-wise thresholds (G2), and slowdown attack surfaces on the real-traffic SR-BH 2020 dataset under pre-registered criteria using single-thread CPU (batch 1) end-to-end latency and real-user benign traffic. Experiments show that joint-trained internal early exit in char-CNN achieves **3.5–3.6**× speedup at stand-alone accuracy via model compression rather than deferral, while the char-CNN → DistilBERT cascade defers only 0.7–0.8% of test requests to inherit **56–74%** of DistilBERT's SQLi-to-CmdI error reduction and achieves **10.6–12.3**× pooled speedup (**23.8–29.3**× with the joint-trained exit as the first stage) over stand-alone DistilBERT. However, its Macro-F1 loss meets the pre-specified 0.30 pp tolerance only on seed 42 (**−0.03 pp**) and fails to replicate on confirmation seed 43 (**−0.32 pp**), class-wise thresholds fail to replicate on new seeds, and single URL encoding inflates expected per-attack latency by **6.4**× while inducing first-stage evasion. Ultimately, cascade effectiveness is governed by stage latency ratios and first-stage capacity rather than threshold tuning, requiring joint reporting of end-to-end latency, deployment base rates, real-user benign deferral, and multi-seed variance.

Keywords: Web attack detection, Cascade inference, Character-level CNN, DistilBERT, Command injection, Inference latency

I. 서론

웹 애플리케이션 방화벽(web application firewall, WAF)은 모든 인바운드 HTTP 요청의 인라인 경로에 놓이므로, 분류 정확도뿐 아니라 요청 한 건을 처리하는 지연 시간을 함께 충족해야 한다. 동시에 보안 분야의 기계학습 평가에서는 실험실 측정 조건과 실제 배포 환경의 괴리가 성능을 과장한다는 점이 반복해 지적되어 왔다[1].

본 연구는 수동 특징 공학(n-gram 빈도 사전 등)에 의존하지 않고 원시 바이트열이나 서브워드 토큰을 종단간(end-to-end) 학습하는 딥러닝 웹 공격 탐지기를 대상으로 한다. 다중 웹 공격 분류에서 드러나는 핵심 병목은 정상 대 공격의 이진 차단보다, 세미콜론(;), 파이프(|), 따옴표(') 등 국소 구분자를 공유하는 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI) 사이의 공격 종류 오탐이다. 문자 수준 합성곱 신경망(character-level convolutional neural network, char-CNN)[2]은 단일 스레드 CPU에서 평균 0.35~0.60ms 안에 요청을 처리하지만 짧은 합성곱 창 때문에 SQLi를 CmdI로 오분류하는 경향이 남고, 사전학습 트랜스포머인 DistilBERT[3]는 전역 어텐션으로 이 혼동을 크게 줄이는 대신 약 45~46ms가 걸려 단독 인라인 배치가 부담스럽다.

이 속도–정확도 간극을 메우기 위해 가벼운 1차 모델이 대다수 요청을 즉시 판정하고 확신도가 임계값 τ 미만인 모호한 요청만 무거운 2차 모델로 넘기는 캐스케이드(cascade)[4]와 내부 조기 종료(early exit)[5]가 제안되어 왔다. 그러나 기존 웹 보안 캐스케이드 연구들에서는 네 가지 핵심 질문이 통합적으로 검증되지 않았다. 첫째, 토큰화와 바이트 인코딩을 포함한 단일 스트림(배치 1) 종단간 공정 지연에서도 이론적 가속이 실현되는가? 둘째, 공격 요청이 대다수인 벤치마크 시험 집합이 아니라 낮은 실제 배포 공격 기저율(π = 3.3%)[6]과 취약점 스캐너가 아닌 실사용자 정상 트래픽 분포에서도 넘김 비용이 낮게 유지되는가? 셋째, 1차 확신도 기반의 넘김 판단(단일 임계값 및 예측 클래스별 임계값)[7]이 다중 분할 seed에서도 일관된 이득을 내는가? 넷째, 공격자가 페이로드를 변형해 2차 호출을 유도하는 비용 기반 지연 유발 공격(slowdown attack)[8]에 얼마나 노출되는가?

실트래픽 기반 다중 공격 데이터셋인 SR-BH 2020[9] 위에서 수행한 본 논문의 핵심 기여는 세 가지이다.

- 딥러닝 캐스케이드 및 조기 종료 **4종(DL-B·DL-C·DL-A·E14)**의 속도–정확도 교환 규명. 동일 5-fold 분할과 동일 단일 스레드 CPU에서 네 구성을 비교하고, 선행 연구(WAMM[10])의 DistilBERT를 재구현해 기준선으로 삼는다. char-CNN 내부 조기 종료(DL-C)는 넘김 판단이 아닌 결합 학습 보조 헤드의 모델 압축 효과로 두 seed 모두 **3.5~3.6**배 가속을 달성했고, char-CNN → DistilBERT(DL-A)는 합산 **10.6~12.3**배 가속(결합 학습 출구를 1차로 결합한 E14는 **23.8~29.3**배)을 얻되 기본 seed 42(**−0.03%p**)와 확인용 seed 43(**−0.32%p**) 사이에서 사전 정확도 허용폭(0.30%p) 충족이 갈리는 재현 한계를 정량화한다.
- 실사용자 정상 트래픽 분리 평가와 넘김 판단의 구조적 한계 규명**(A2·E13).** 단일 임계값(B0)이 공격 쪽 F1을 맞추려 임계값을 올릴 때, 1차가 정상으로 맞게 예측했으나 확신도가 덜한 실사용자 정상 요청이 대량 넘어가(r_N = 33.6%) 특정 폴드 배수를 떨어뜨리는 구조적 비용(A2)을 진단한다. 나아가 이를 완화하려던 예측 클래스별 임계값(G2)이 새 seed 44·45 확인 실험(E13)에서 재현되지 않음(유의성 0/2)을 보여, 넘김 임계값 조정이 일관된 해결책이 아님을 입증한다.
- **SQLi→CmdI** 오탐 분해**,** 원 논문**(128** 토큰**)** 문맥 잘림 검증**(E12)** 및 비용 공격면**(E15)** 규명**.** DistilBERT의 실질적 기여가 공격 차단율 상승이 아니라 SQLi→CmdI 오탐 제거에 있으며 캐스케이드가 0.7~0.8% 넘김만으로 이 감소분의 **56~74%**를 회수함을 밝힌다. 또한 원 논문 설정(128 토큰)이 긴 주입 요청을 잘라내어 char-CNN 단독보다도 부정확해짐(−1.70~−1.96%p)을 보이고, URL 인코딩 1회 변형으로 공격 요청당 기대 지연이 **6.4**배 증가하며 1차 탐지 회피가 발생하는 공격면을 실측한다.

결과에 맞춘 사후 기준 변경(HARKing)을 막기 위해 모든 확인 실험의 판정 규칙은 측정 전 버전 관리에 고정하였으며(커밋 2a456d7~d9d3678), 기준 외 분석은 사후 분석 또는 병기로 구분한다.

II. 관련 연구

웹 공격 탐지 연구는 HTTP 요청 문자열을 1차원 시퀀스로 다루며[11], Zhang 등[2]의 char-CNN은 별도 토크나이저 없이 원시 바이트에서 지역 n-gram 패턴을 추출하는 경량 신경망으로 널리 쓰인다. Sureda Riera 등[9]이 공개한 SR-BH 2020은 실운영 웹 서버 앞단의 ModSecurity 로그를 검수해 CAPEC 공격 분류를 붙인 다중 클래스 데이터셋이다. 이 데이터셋 위에서 Osama 등[10]은 중복 제거와 정제를 거친 WAMM 프레임워크를 통해 DistilBERT[3]의 성능을 보고하였고, 이중 채널 BERT(WADBERT)[12]와 라벨 정제 기반 이상 탐지(X-WAD)[13]가 후속 연구로 제안되었다. 본 연구는 Osama 등[10]의 딥러닝 최고 모델인 DistilBERT를 동일 분할·동일 CPU에서 512 토큰 및 원 설정(128 토큰)으로 재구현해 직접 비교한다.

캐스케이드 추론은 최대 소프트맥스 확률(maximum softmax probability, MSP)[4][14] 임계값으로 경량 1차와 대형 2차를 잇는 구조로, Tasdemir 등[6]이 SQLi 단일 공격 탐지에서 BERT 대비 약 20배 가속을 보고하였다. 어느 요청을 넘길지 결정하는 문제는 선택적 분류[15] 및 넘김 학습[16]과 맞닿아 있는데, Enomoto와 Eda[17]는 온도 보정[18]이 캐스케이드 효율을 떨어뜨릴 수 있음을 보였고, Jitkrittum 등[7]은 최적 넘김 규칙이 1·2차 정답 확률 차이에 의해 결정되므로 2차가 특정 하위 분포에만 강할 때 1차 확신도 게이트가 구조적으로 한계를 가짐을 증명하였다. 단일 신경망 내부에 보조 분류기를 두는 조기 종료(BranchyNet)[5]는 전처리 중복을 없앤 대안이다. 한편 적응형 추론은 조기 종료를 방해해 연산 비용을 폭증시키는 지연 유발 공격(DeepSloth)[8]에 노출되며, 인라인 보안 장비 평가에서는 단일 스트림 지연 분리 측정(MLPerf)[19]과 낮은 실제 공격 기저율(base rate)[20] 반영이 필수적이다.

III. 방법

3.1 입력 표현과 딥러닝 캐스케이드 구성

각 HTTP 요청에서 URI와 본문(body)을 이어 붙인 원시 바이트열을 $b = (b_0, b_1, ..., b_{L-1})$이라 한다. 문자 수준 모델(**char-CNN**[2] 계열)은 고정 길이 패딩 없이 실제 길이 L 그대로의 1차원 바이트 시퀀스를 읽는다(최대 커널 크기인 7바이트보다 짧은 경우만 7바이트까지 채움). 각 바이트(257개 기호)를 임베딩한 뒤 1차원 합성곱과 시간축 최대 풀링을 수행해 선형 분류기에 넣는다. 사전학습 트랜스포머인 **DistilBERT**[3]는 원문 문자열을 WordPiece 토크나이저(소문자화 포함)로 분절해 입력받는다.

본 연구는 경량 기준선인 **char-CNN**(임베딩 64차원, 커널 3·5·7 각 128채널 병렬, 141,252 파라미터)과 고비용 참조 모델인 **DistilBERT**(`distilbert-base-uncased`, 66.96M 파라미터) 사이에서 속도–정확도 교환을 달성하기 위해 네 가지 딥러닝 구성을 설계·비교한다(Table 1).

1. **DL-B** (`char-CNN-tiny` → `char-CNN`): 임베딩을 32차원으로 줄이고 커널 3 분기 하나(64채널, 14,692 파라미터)만 남긴 초경량 모델을 1차로, 전체 char-CNN을 2차로 잇는 캐스케이드이다.
2. **DL-C** (`char-CNN-EE`, 내부 조기 종료): 별도 1차 모델 없이 char-CNN의 커널 3 분기 풀링 출력(128차원) 뒤에 보조 선형 분류기(128→4)를 추가한 구조이다(141,768 파라미터). 학습 시 주 헤드 손실과 보조 헤드 손실을 $L = L_{main} + 0.3\ L_{aux}$로 결합해 동시에 최적화한다. 추론 시에는 임베딩과 커널 3 분기만 먼저 계산해 보조 헤드 확신도가 임계값 이상이면 즉시 종료하고, 미달할 때만 커널 5·7 분기를 마저 계산한다. 보조 헤드 경로와 동일한 크기(임베딩 64, 커널 3 하나, 128채널, 41,668 파라미터)를 처음부터 혼자 학습한 대조 모델을 **char-CNN-k3**라 한다.
3. **DL-A** (`char-CNN` → `DistilBERT`): 전체 char-CNN을 1차로 내리고 미세조정된 DistilBERT를 2차로 두는 2단계 캐스케이드이다. 원 논문(WAMM[10])은 최대 길이를 128 토큰으로 두었으나, 본 데이터셋의 학습+검증 풀에서 128 토큰 제한은 전체의 6.9%(15,553행), 특히 길이가 긴 CodeInjection의 24.7%와 CmdI의 26.5%를 잘라내어 참조 모델에 불리하게 작용한다. 따라서 주 실험(E10·E11)에서는 잘림이 0%인 **512** 토큰(토큰 수 중앙값 54, p99 189)을 사용해 참조 모델의 상한 성능을 보존하고, 원 논문의 **128** 토큰 설정(`DistilBERT-128`)은 E12(5.3절)에서 별도로 대조한다.
4. **E14** (`k3 exit` → `DistilBERT`): DL-C에서 결합 학습된 커널 3 보조 헤드 출구만

떼어내어 1차로 삼고, DistilBERT(512 토큰)를 2차로 연결해 1차 추론 비용까지 줄인 결합 캐스케이드이다.

Table 1. Deep-learning models compared in 5-fold cross-validation

Model	Role & setup	Input	Params
char-CNN	Ref.; DL-B 2nd; DL-A 1st	raw bytes (L)	141,252
char-CNN-tiny	DL-B 1st stage (k3, 64-ch)	raw bytes (L)	14,692
char-CNN-k3	Single-branch solo control	raw bytes (L)	41,668
char-CNN-EE	DL-C early exit; E14 1st	raw bytes (L)	141,768
DistilBERT	DL-A/E14 2nd; WAMM[10] ref.	≤512 (or 128) tok.	66.96M

Notes: All models use class-weighted cross-entropy on the same 5 folds (early stopping patience 5 on internal validation Macro-F1; DistilBERT fine-tuned for 3 epochs with Adam, lr 2e-5, batch 16).

3.2 넘김 판단(B0·G2)과 배포 기저율 재가중 합산 배수

1차 모델 h_1이 요청 x에 대해 출력한 소프트맥스 확률을 $p_1(k|x)$, 최대 소프트맥스 확률을 $s(x) = \max_k p_1(k|x)$라 할 때, 임계값 τ에 따른 2단계 캐스케이드의 최종 예측 $\hat{y}(x)$는 다음과 같다.

$$\hat{y}(x) = h_1(x) \text{ if } s(x) \geq \tau, \text{ otherwise } h_2(x) \quad (1)$$

기본 단일 임계값 게이트**(B0)**는 각 폴드의 내부 검증 집합에서 검증 Macro-F1이 2차 단독 검증 Macro-F1의 0.5%p 이내를 유지하는 지점 중 배포 기대 넘김률(식 3)이 최소인 τ를 골라 시험 폴드에 1회 적용한다. 이에 대조하는 예측 클래스별 임계값**(G2)**은 클래스마다 확신도 분포와 오분류 비용이 다른 점을 반영해, 1차 예측 클래스 $\hat{c}_1(x) = \text{argmax}_k\, p_1(k|x)$별로 서로 다른 임계값 τ_c를 둔다.

$$\text{defer } x \Leftrightarrow s(x) < \tau_{\hat{c}_1(x)},\ \hat{c}_1(x) = \text{argmax}_k\, p_1(k|x) \quad (2)$$

임계값 벡터 $(\tau_1, ..., \tau_4)$는 내부 검증 집합에서 전체 넘김 예산을 0.5%p씩 늘릴 때 단위 넘김당 Macro-F1 상승폭이 가장 큰 클래스에 예산을 배정하는 탐욕적 경로 탐색으로 정한다(사전 예비 실험에서 검토한 온도 보정·마진·엔트로피 신호 교체, 학습형 라우터, 마할라노비스 등 특징 공간 거리 점수는 모두 B0 대비 과잉 넘김을 유의하게 줄이지 못해 본문에서는 B0와 G2를 중심으로 보고한다).

1차 지연을 t_1, 2차 지연을 t_2, 비교 기준 단독 모델 지연을 t_{ref}라 하자(DL-C는 앞단 커널 3 출구 시간을 t_{front}, 뒷단 커널 5·7 시간을 t_{rest}라 할 때 $t_1 = t_{front}$, $t_2 = t_{rest}$, t_{ref} = char-CNN 지연; DL-A는 t_{ref} = DistilBERT 지연 t_R). 넘김률이 정상 요청(r_N)과 공격 요청(r_A)에서 크게 다르므로, 배포 환경의 공격 기저율 $\boldsymbol{\pi}$ = **3.3%**[6]를 반영한 기대 넘김률 E_π와 폴드 f의 기대 지연 t_f는 다음과 같다.

$$E_\pi = \pi \cdot r_A + (1 - \pi) \cdot r_N,\ t_f = t_1 + E_\pi \cdot t_2 \quad (3)$$

교차검증의 5개 폴드에서 폴드별 속도 배수 $S_f = t_{ref} / t_f$를 단순 산술평균하면, 넘김이 많아 느려진 폴드의 비용이 가려져 배포 속도를 과대평가하게 된다. 따라서 본 논문은 다섯 폴드의 평균 비용으로 계산한 합산 속도 배수 $\bar{S}$**(**폴드 배수 $\mathbf{S_f}$의 조화평균**)**를 헤드라인 지표로 쓰고 산술평균은 괄호에 병기한다.

$$\bar{S} = t_{ref} / \left((1/5) \Sigma_f t_f \right) \quad (4)$$

IV. 실험 환경

4.1 데이터셋과 실사용자 정상 층 분리

SR-BH 2020[9] 원본 907,815행의 CAPEC 라벨을 정상(Normal), SQL 삽입(SQLi), 코드 삽입(CodeInjection, 서버 측 PHP 실행 탐침 위주), 명령 삽입(CmdI)의 4클래스로 매핑하였다. 정규표현식 감사로 적발된 정상 라벨 내 공격 구문 의심 행(30,280건), 동일 문자열 다중 라벨 충돌 행, 완전 중복 행을 제거해 총 **265,939**행(학습 186,157 · 검증 39,891 · 봉인 시험 39,891; 공격 비율 74.8%)을 구성하였다. 입력은 URI와 본문(F2)만 사용한다(쿠키를 추가한 F3은 쿠키 값을 마스킹한 대조군 F3M과 보정 p = 0.17로 유의한 차이가 없어 식별자 과적합을 배제함). 최소 클래스인 CmdI는 전체의 **2.1%**(학습+검증 풀 3,886건)에 불과하다.

데이터 분석에서 확인된 핵심 특성은 정상(Normal) 요청의 출처 편향이다. 중복 제거 후 정상 요청의 **97.8%**가 취약점 스캐너(Nikto, OpenVAS 등)의 자동 크롤링 요청이었고, 일반 브라우저 User-Agent로 들어온 실사용자 정상 요청은 학습+검증 풀**(226,048**행**)** 내 **550**건**(**폴드당 약 **110**건**)**뿐이었다. 실제로 char-CNN 기준 실사용자 정상 층의 오탐률(false positive rate, FPR)은 **2.73%**로 스캐너 정상 층(0.36%)의 **7.6**배였으며, 1차 확신도 또한 실사용자 정상 요청에서 훨씬 낮게 형성되어 넘김률 r_N이 스캐너 위주 검증 분포보다 20%p 이상 높게 나타났다. 따라서 스캐너 위주의 전체 정상 지표로 평가하면 오탐과 넘김 비용을 크게 과소평가하게 되므로, 모든 평가에서 실사용자 정상 층과 스캐너 층을 분리하고 주 판정은 실사용자 정상 $\mathbf{r_N}$ 기준으로 수행한다.

4.2 평가 절차와 사전 고정 판정 기준

다중 클래스 불균형을 반영해 주 분류 지표는 클래스별 F1의 비가중 평균인 **Macro-F1**을 사용한다. 모든 실험은 봉인된 시험 집합(39,891행)을 제외한 학습+검증 풀 **226,048**행의 동일한 층화 5-fold 교차검증에서 수행한다(각 폴드 내부 학습 부분을 9:1로 다시 나누어 조기 종료 및 임계값 τ 선택용 내부 검증 집합으로 사용). 폴드 간 차이는 학습 집합 중복을 교정하는 Nadeau–Bengio 보정 대응 t-검정[21]과 Holm 보정으로 검정한다. 분할 및 초기화 seed는 42를 기본 판정에, 43(G2 추가 확인은 44·45)을 사전 등록된 재현성 확인에 사용한다.

추론 지연은 단일 데스크톱(AMD Ryzen 7 9800X3D, RAM 32GB, RTX 4080 SUPER, Windows 11, PyTorch 2.6.0)에서 인라인 WAF의 단일 스트림 처리를 반영하여 **CPU** 단일 스레드**(1 thread)**·배치 **1** 조건에서 원문 입력부터 전처리(바이트 인코딩·토큰화)와 순전파를 거쳐 예측이 나오기까지의 종단간 시간을 잰다(2,000건 × 20회 워밍업 후 30회 반복 중앙값 p50)[19]. 실행 간 클럭 표류(4~7%)를 통제하기 위해 비교 모델들은 반드시 같은 실행 안에서 요청마다 번갈아 측정해 같은 실행 값끼리만 배수를 계산한다.

기준 단독 모델 대비 우위("위") 사전 판정 기준은 측정 전에 다음 세 조건을 모두 충족하는 것으로 고정하였다(실사용자 정상, $\pi = 3.3\%$): ① 합산 배수 $\bar{\mathbf{S}} \geq \mathbf{1.2}$**배,** ② **5**개 폴드 모두 $\mathbf{S_f > 1.0}$**배,** ③ 기준 모델 대비 **Macro-F1** 평균 손실 ≤ **0.30%p(**$\mathbf{\Delta F1 \geq -0.30\%p}$**)**.

V. 실험 결과

5.1 딥러닝 캐스케이드(DL-B · DL-C · DL-A · E14)의 속도–정확도 교환

본 학습에 앞서 수행한 층별 지연 분해 관문 실험(E7)에서 char-CNN 추론 시간의 **91.0%**가 세 합성곱 층**(**커널 **3·5·7)**에서 소비되고 바이트 인코딩과 호출 고정비는 **8.5%**에 불과함이 확인되어, 커널을 줄이거나 앞 분기만 실행하는 구조의 물리적 가속 가능성을 확인하였다. 또한 2차 후보인 재구현 DistilBERT(512 토큰) 단독 모델은 5-fold 교차검증에서 Macro-F1 **0.9914**(seed 42)를 기록해 char-CNN 단독**(0.9884)**을 넘어 관문(E10-1)을 통과하였다. Table 2와 Fig. 1은 네 가지 딥러닝 구성의 본 측정 및 확인 실험 결과를 종합한 것이다.

Table 2. Speed–accuracy trade-off of deep-learning cascades vs. stand-alone references (5-fold CV, CPU batch 1, 1 thread, end-to-end unpadded latency, real-user benign traffic at π = 3.3%)

Configuration	Reference R (F1)	Seed	Gate	Macro-F1 (ΔR)	r_N / r_A	Pooled $\bar{S}$ (arith.*)	Fold S_f range	Verdict vs. R
DL-B: charcnn_tiny → char-CNN	char-CNN (0.9884)	42	B0	0.9839 (−0.45 pp)	30.2% / 4.0%	2.13× (2.39×*)	1.52–4.10×	No (F1 loss)
DL-C: charcnn_ee (internal exit)	char-CNN (0.9884)	42	B0	0.9874 (−0.10 pp)	1.1% / 0.4%	**3.52**× (3.53×*)	3.24–3.62×	**Yes**
DL-C: charcnn_ee (confirm.)	char-CNN (0.9862)	43	B0	0.9895 (+0.33 pp)	0.0% / 0.005%	**3.62**×	3.62–3.62×	**Yes (2/2)**
Control: charcnn_k3 (solo)	char-CNN (0.9884)	42	—	0.9748 (−1.36 pp)	0.0% / 0.0%	3.62×	3.62–3.62×	No (F1 loss)
char-CNN solo vs. DistilBERT	DistilBERT (0.9914)	42	—	0.9884 (−0.303 pp)	0.0% / 0.0%	80.2×	80.2–80.2×	No (0.003 pp short)
DL-A: char-CNN → DistilBERT	DistilBERT (0.9914)	42	B0	0.9911 (**−0.03 pp**)	7.1% / 0.7%	**12.3**× (51.6×*)	2.95–80.2×	**Yes (seed 42 only)**
DL-A: confirm. (E11)	DistilBERT (0.9947)	43	B0	0.9914 (**−0.32 pp**)	8.9% / 0.6%	**10.6**× (38.3×*)	3.93–80.2×	No (p = 0.038)
DL-A + G2 (secondary)	DistilBERT (0.9914)	42	G2	0.9920 (+0.06 pp)	0.7% / 0.7%	51.1× (57.5×*)	32.5–80.2×	Secondary (E13 failed)
DL-A + G2 (secondary)	DistilBERT (0.9947)	43	G2	0.9920 (−0.27 pp)	1.1% / 0.6%	42.5× (50.6×*)	13.7–80.2×	Secondary (E13 failed)
E14: k3 exit → DistilBERT	DL-A B0 (12.2×)	42	B0	0.9883 (−0.28 pp)	—	**29.3**× (×2.40)	—	**Yes (vs. DL-A)**
E14: confirm.	DL-A B0 (10.5×)	43	B0	0.9919 (+0.05 pp)	—	**23.8**× (×2.27)	—	**Yes (2/2 vs. DL-A)**

Notes: Values with * are arithmetic means of fold speedups shown in parentheses. Same-run p50 latencies (never mixed across runs): E8 run has charcnn_tiny t_1 = 0.0606 ms, char-CNN t_2 = 0.3451 ms, k3 front t_{front} = 0.0954 ms; E10 run has char-CNN t_2 = 0.563 ms, DistilBERT(512) t_R = 45.15 ms (80.2×); E12 run (used for E12–E15) has t_2 = 0.604 ms, t_{front} = 0.159 ms, $t_R(512)$ = 46.10 ms, $t_R(128)$ = 46.08 ms.

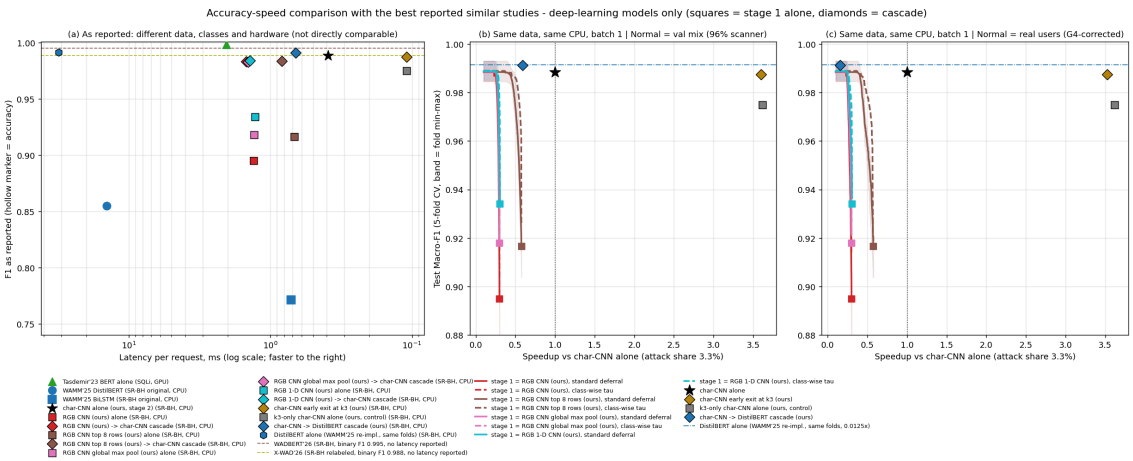


Fig. 1. Accuracy–latency trade-off of deep-learning configurations on SR-BH 2020 (5-fold CV, CPU batch 1, 1 thread). Panel (a) shows end-to-end latency alongside reference points reported by prior work under different hardware/class setups; panels (b) and (c) compare speedup vs. Macro-F1 on the exact same folds under validation benign mix and real-user benign traffic.

1) DL-B (charcnn_tiny → char-CNN**): 1**차 용량 부족으로 인한 탈락**.** 초경량 1차(t_1 = 0.0606ms)는 단독 Macro-F1이 0.9601로 낮아 실사용자 정상 요청의 30.2%를 넘겼고, 합산 배수는 **2.13**배(산술 2.39배)에 그쳤으며 Macro-F1이 0.9839로 char-CNN 단독 대비 **−0.45%p** 하락해 사전 기준(손실 0.30%p 이내)을 넘지 못했다.

2) DL-C (charcnn_ee**):** 넘김 판단이 아닌 결합 학습 기반 **2**차 모델 압축 효과**.** char-CNN 내부의 커널 3 분기에 보조 헤드를 결합 학습한 charcnn_ee는 seed 42에서 합산 **3.52**배(산술 3.53배, F1 0.9874, 단독 대비 −0.10%p), 확인용 seed 43에서 합산 **3.62**배(F1 0.9895, 단독 대비 +0.33%p)를 기록해 두 seed 모두 char-CNN 단독과 같은 정확도를 **3.5~3.6**배 빠르게 내며 사전 기준을 통과하였다. 그러나 폴드별 운영점을 보면 seed 42의 다섯 폴드 중 셋, seed 43의 전 폴드가 넘김률 **0%**였다(보조 헤드 단독 F1이 이미 0.9849). 즉 이득의 원천은 넘김 판단이 아니라 보조 헤드 자체였다. 같은 크기를 처음부터 혼자 학습한 대조 모델 charcnn_k3(seed 42, F1 **0.9748**, char-CNN 대비 −1.36%p)는 보조 헤드보다 1.01%p 낮아 기준을 통과하지 못했다(보정 p = 0.34; 최적 에폭은 seed 43 보조 헤드가 12~19였고, 같은 seed 42에서 비교하면 보조 헤드는 8·10·2·19·3, k3 단독은 3·20·9·6·2였다). 조기 종료 인내(patience)를 늘린 단독 대조 실험(E16)은 수행하지 않았으므로, +1.01%p 차이가 결합 학습의 표현력 몫인지 단독 모델의 이른 조기 종료(최적화 안정성)인지는 가리지 못했다. 따라서 DL-C는 넘김 판단 효과가 아닌 **2**차 모델 압축**(**결합 학습한 **k3** 출구**)**으로 해석한다.

3) DL-A (char-CNN → DistilBERT**): 10.6~12.3**배 가속과 **seed 43** 미재현 경계**.** E10 실행에서 char-CNN(0.563ms)과 DistilBERT(45.15ms)의 지연 격차는 **80.2**배였다. 가장 빠른 끝단인 char-CNN 단독은 80.2배 빠르나 DistilBERT 대비 F1이 seed 42에서 −0.303%p(0.9884 대 0.9914)로 기준(−0.30%p)에 0.003%p 미달했다. 반면 char-CNN → DistilBERT 캐스케이드(B0)는 공격 요청의 0.6~0.7%, 실사용자 정상 요청의 7.1~8.9%만 넘기면서 실사용자 정상 기준 합산 배수로 재구현 DistilBERT 단독보다 **seed 42**에서 **12.3**배**(**산술 **51.6**배**,** 평균 **3.67ms), seed 43**에서 **10.6**배**(**산술 **38.3**배**,** 평균 **4.28ms)** 빨랐다(두 seed 모두 다섯 폴드 전부 1배 초과; char-CNN 단독 대비로는 지연 6.5~7.6배에 F1 +0.28·+0.53%p이나 보정 p = 0.34·0.23으로 유의성 미확인). 그러나 DistilBERT 단독 대비 F1 손실은 **seed 42**에서 **−0.03%p(0.9911** 대 **0.9914), seed 43**에서 **−0.32%p(0.9914** 대 **0.9947,** 보정 **p = 0.038)**였다. 즉 사전 기준(손실 0.30%p 이내)은 **seed 42** 한정으로만 충족했고, 확인용 seed 43에서는 0.02%p 차로 재현되지 않았다. seed 43 실패의 내용을 보면 캐스케이드 F1은 0.9911 → 0.9914로 그대로였고, 기준인 DistilBERT 단독이 3 에폭 미세조정의 seed 간 편차로 인해 0.9914 → 0.9947로 올라갔다. 캐스케이드는 트래픽의 약 93%를 1차 char-CNN 판정으로 끝내므로 2차 단독이 강해져도 상한이 묶인다.

4) E14 (k3 exit → DistilBERT**): 1**차 경량화를 통한 추가 가속**.** DL-A의 1차를 char-CNN 전체(0.604ms, E12 실행) 대신 결합 학습한 k3 출구(0.159ms)로 바꾸면, 같은 2차(DistilBERT, 46.10ms)에서 합산 배수가 **seed 42**에서 **29.3**배**(DL-A 12.2**배 대비 **×2.40, F1 0.9883, DL-A** 대비 **−0.28%p)**, **seed 43**에서 **23.8**배**(DL-A 10.5**배 대비 **×2.27, F1 0.9919, DL-A** 대비 **+0.05%p)**로 커져 두 seed 모두 DL-A 대비 사전 기준을 충족하였다. 단, seed 42의 −0.28%p는 허용 경계이고 최대 289.8배 폴드는 넘김이 0%여서 이득의 상당 부분은 1차가 4배 싸진 것(0.159 대 0.604ms)이며, DistilBERT 단독 대비로는 −0.31·−0.28%p이므로 DL-A 대비 판정이다.

5.2 넘김 판단의 한계: 느린 폴드 진단(A2)과 클래스별 τ 재확인(E13)

단일 임계값**(B0)**의 구조적 비용**(A2,** 사후 기술**).** seed 42 DL-A에서 합산 배수(12.3배)가 산술평균(51.6배)보다 크게 낮았던 원인은 폴드 4의 배수가 **2.95**배(다른 폴드는 46.9~80.2배)였기 때문이다. 폴드 4에서는 검증 F1 목표를 맞추려 검증 넘김 예산이 3.0%(τ = 0.985)까지 올라갔고, 이때 실사용자 정상 넘김률 r_N이 **33.6%**(Wilson 95% 구간 [25.4, 43.0], n = 107; 내부 검증에서도 28.9%)로 커졌다. 넘긴 실사용자 정상 36건 중 **34**건은 **1**차가 이미 **Normal**로 맞게 예측했으나 확신도가 덜한 요청이었다. 즉 단일 τ는 공격 쪽 F1을 맞추려 τ를 올리면 실사용자 정상 요청이 대량 넘어가는 구조적 비용이 있다.

클래스별 임계값**(G2)**의 새 **seed 44·45** 확인 실패**(E13).** 이 구조적 비용 때문에 seed 42·43에서 예측 클래스별 임계값(G2)을 병기했을 때는 실사용자 정상 넘김이 억제되어 합산 배수가 51.1배·42.5배로 크게 뛰었다(Table 2). 이 이득이 일반화되는지

검증하기 위해 측정 전 사전 등록한 새 **seed 44·45** 확인 실험**(E13)**(채택 조건: ① 합산 배수 이득 $\bar{S}(G2)/\bar{S}(B0) - 1 \geq 20\%$, ② G2 F1 ≥ B0 − 0.30%p, ③ 폴드별 E_π 차이 보정 $p < 0.05$)을 수행한 결과, 두 **seed** 모두 세 조건을 채우지 못했다.

- **seed 44**: 합산 배수 B0 **76.0**배 → **G2 59.6**배 **(−21.6%,** 조건 ① 미달**)**, F1 0.9908 → 0.9920(조건 ② 충족), E_π 차 보정 $p = 0.34$**(**조건 ③ 미달**)**.
- **seed 45**: 합산 배수 B0 **42.4**배 → **G2 54.4**배 **(+28.4%,** 조건 ① 충족**)**, F1 0.9903 → 0.9913(조건 ② 충족), E_π 차 보정 $p = 0.54$**(**조건 ③ 미달**)**.

원인을 분석하면, G2의 이득은 B0가 실사용자 정상을 많이 넘기는 나쁜 운영점을 고른 폴드(seed 45 폴드 1: 17.6→32.5배, seed 42 폴드 4와 같은 유형)를 메우는 데서만 생겼다. 반면 seed 44의 B0는 모든 폴드에서 이미 상한(76.3배)에 붙어 있어 G2가 줄일 비용이 없었고 오히려 두 폴드를 깎았다. 즉 seed 42·43에서 관찰한 클래스별 τ의 큰 이득은 새 seed 44·45에서 재현되지 않았으며(기준 이상 1/2 seed, 유의성 0/2), 넘김 판단 규칙을 바꾸는 시도는 일관된 이득을 내지 못했다. (한편 E13 과정에서 병기된 DL-A B0 대 동일 seed DistilBERT 단독의 F1 차이는 seed 44 +0.08%p, seed 45 −0.24%p로 네 seed 중 세 seed에서 DistilBERT 단독과 0.30%p 이내였으나, 사전 판정 seed 43 미달은 그대로 보고한다.)

5.3 왜 DistilBERT인가: CmdI 오답 분해(사후 분석)와 128 토큰 재구현(E12)

char-CNN보다 80배 무거운 DistilBERT가 실제로 교정하는 오류를 규명하기 위해, 5-fold 합산 226,048행의 오류 분해(사후 분석)와 원 논문(WAMM[10])의 128 토큰 설정 재구현 실험(E12)을 수행하였다(Table 3).

Table 3. Pooled 5-fold error decomposition (226,048 rows) comparing stand-alone char-CNN, stand-alone DistilBERT (512 tokens, post-hoc), and the original 128-token DistilBERT setup (E12)

Metric / error count	s42 char-CNN	s42 DistilBERT(512)	s42 tok128 (E12)	s43 char-CNN	s43 DistilBERT(512)	s43 tok128 (E12)
Overall Macro-F1	0.9884	0.9914	0.9718 (−1.96 pp)	0.9862	0.9947	0.9777 (−1.70 pp)
Missed attack (Attack → Normal)	791	736 (−7.0%)	—	839	777 (−7.4%)	—
Benign false alarm (Normal → Attack)	206	245	—	140	118	—
Real-user / scanner FPR	2.00% / 0.35%	1.82% / 0.43%	—	1.27% / 0.24%	1.09% / 0.21%	—
Attack-type confusion	**266**	**167 (−37.2%)**	—	**359**	**59 (−83.6%)**	—
└ SQLi → CmdI errors	**192**	**99 (−48.4%)**	**847**	**252**	**2 (−99.2%)**	**551**
CommandInjection F1	**0.9682**	**0.9784**	**0.9038**	**0.9618**	**0.9903**	**0.9284**

1) SQLi → CmdI 오답 제거와 캐스케이드의 회수율**(**사후 분석**).** Table 3에서 보듯 공격을 정상으로 놓친 차단 실패는 char-CNN에서 DistilBERT(512)로 가도 두 seed 모두 약 **7%** (791→736건, 839→777건)만 줄었고, 실사용자 정상 FPR(n = 550)도 2.00%→1.82%(seed 42), 1.27%→1.09%(seed 43)로 요청 1건 차이에 불과했다. 즉 DistilBERT의 이득은 공격 차단이 아니라 공격 종류 구분**,** 특히 **SQLi**를 최소 클래스인 **CmdI**로 오분류하던 오답의 제거**(seed 42 192→99**건으로 **−48.4%, seed 43 252→2**건으로 **−99.2%)**에서 나왔으며, 이를 통해 CmdI F1을 0.9682→0.9784, 0.9618→0.9903으로 끌어올렸다. 구조적 가설로, char-CNN의 3·5·7바이트 합성곱과 최대 풀링은 ;·|·' 같은 짧은 구분자에 국소 반응하는 반면 DistilBERT는 512 토큰 전체 어텐션으로 구분자 뒤가 SQL 절인지 운영체제 명령인지 본다. 전체 요청에서 char-CNN 확신도가 0.99 이상인 비율은 97.0%이지만, DistilBERT가 고친 오답 집합에서 char-CNN 확신도 중앙값은 **0.775(seed 42)·0.766(seed 43)**으로 낮았고 확신도 ≥ 0.99 비율은 **15.1%·10.7%**에 그쳤다. 덕분에 캐스케이드**(B0)**는 시험 요청의 **0.7~0.8%**만 넘기면서 **SQLi→CmdI** 오탐을 **seed 42**에서 **192→123**건**(**단독 **99**건까지 감소분의 **74%), seed 43**에서 **252→113**건**(**단독 **2**건까지 감소분의 **56%)**으로 줄여 이 감소분의 절반 이상을 물려받았다(사후 분석; 캐스케이드 차단 실패도 697·743건으로 단독 736·777건보다 적음). 반대로 char-CNN이 ≥

0.99로 과신하며 틀리는 나머지 11~15%는 단일 임계값으로 넘기지 못해 캐스케이드의 정확도 상한을 묶는다.

2) 원 논문 설정**(128** 토큰**)** 재구현 비교**(E12).** 원 논문(WAMM[10]) 설정인 128 토큰으로 재구현한 DistilBERT는 512 토큰보다 유의하게 낮았으며(seed 42 **0.9718** 대 **0.9914**로 **−1.96%p**, seed 43 **0.9777** 대 **0.9947**로 **−1.70%p**, 둘 다 보정 $p = 0.006$, E12-1 채택), 이 데이터에서 char-CNN 단독(0.9884·0.9862)보다도 낮았다. 손실은 128 토큰에서 잘린 15,553행에 집중되었다(잘린 행 정답률: seed 42 0.994→0.957, seed 43 0.996→0.956; 안 잘린 행은 0.1%p 이내). 특히 주입 구문이 긴 CmdI F1이 0.9784→0.9038·0.9903→0.9284로 급락하고 SQLi→CmdI 오탐이 99→847건·2→551건으로 폭증했다(E12-2). 한편 배치 1에서는 전체 요청의 93%가 원래 128 토큰 이하라 추론 지연이 512 토큰과 거의 같았다(46.10 대 46.08ms). 결과적으로 `char-CNN → DistilBERT-128` 캐스케이드는 DistilBERT-128 단독 대비 합산 **67.5**배**(F1 +1.66%p, seed 42)·76.3**배**(+0.84%p, seed 43)**로 두 seed 모두 위였으나(E12-3), 이는 검증 운영점이 사실상 넘김 0%여서 **"char-CNN** 단독이 **128** 토큰으로 잘린 **DistilBERT-128**보다 정확하고 **76**배 빠르다**"**를 다시 쓴 것이고, 잘림을 없앤 512 토큰 판본은 seed 43에서 재현되지 않았다(E11).

3) 견고성 점검 요약**(T5 · T10).** 시험 폴드 요청의 66.5%(CmdI는 14.7%)가 학습 폴드에 같은 골격 템플릿을 두나, 동일 템플릿이 학습과 시험에 나뉘지 않게 묶음 분할(T5)을 수행해도 char-CNN Macro-F1은 +0.10%p(0.9884→0.9894)로 템플릿 암기에 의한 부풀림(기준 −2.0%p)에 해당하지 않았다. 또한 4개 seed(42~45, T10)에서 char-CNN 단독의 표준편차는 0.18%p(0.9862~0.9901)로 안정적이었으나, 3 에폭 미세조정 DistilBERT의 seed 간 편차(+0.33%p)가 허용폭(0.30%p)과 맞먹어 DL-A의 seed 간 판정 경계를 갈랐다.

VI. 논의 및 한계

실무 배포 관점의 선택 지침**.** 본 논문의 결과는 인라인 WAF 운영 환경에 따라 명확한 설계 기준을 제시한다. 1. 이진 차단**(**정상 대 공격**)** 위주이거나 약 **0.3~0.8%p**의 **Macro-F1** 차이가 허용되는 환경이라면, 무거운 트랜스포머 캐스케이드 없이 **char-CNN** 단독 또는 결합 학습된 내부 조기 종료 모델**(DL-C, k3** 보조 출구**)**을 배치하는 것이 DistilBERT보다 약 80~280배 빠르면서 차단 실패율도 거의 차이가 없다. 2. 관제 자동화를 위해 **SQLi**와 **CmdI** 등 세밀한 공격 종류 분류 정확도가 필수적인 환경이라면 512 토큰 DistilBERT가 SQLi→CmdI 오탐을 줄이는 데 기여하며, 이때 `char-CNN → DistilBERT`(DL-A, 합산 10.6~12.3배) 또는 결합 학습 출구를 1차로 둔 `k3 exit → DistilBERT`(E14, 합산 23.8~29.3배)가 평균 지연을 한 자릿수 밀리초 이하로 낮추는 절충안이 된다(단, 미세조정 seed에 따라 최대 0.32%p의 Macro-F1 손실이 발생할 수 있다).

한계 및 향후 과제**.** 본 연구의 결과는 다음 여섯 가지 한계 안에서 해석되어야 한다. 첫째, 모든 지연은 데스크톱 CPU 1대·단일 스레드·배치 1·PyTorch 구현 그대로에서만 측정하였다. 다중 스레드 CPU 지연(B3 미실시)이나 GPU 배치 서빙에서는 트랜스포머의 병렬화 효율로 t_R / t_1 비율이 줄어 캐스케이드 배수가 작아질 수 있다. 둘째, 교차검증 폴 내 실사용자 정상 요청이 550건(폴드당 약 110건)뿐이어서 실사용자 r_N의 폴드 간 분산이 크다(5.2절 A2에서 특정 폴드 배수가 2.95배로 낮아진 원인). 셋째, DistilBERT 재구현은 원 논문(WAMM[10])의 128 토큰 대신 잘림이 없는 512 토큰과 Adam을 주 기준으로 사용하였다(원 설정인 128 토큰 판본은 E12에서 별도로 측정하여 문맥 잘림 손실을 확인함). 넷째, 1차가 확신도 0.99 이상으로 틀리고 2차가 맞힌 꼬리 오답 110건(대조군 110건)을 원문 감사했을 때 70.9%(대조군 31.8%)가 라벨 불일치로 분류되었으나, 이는 대형 언어모델 1인 판정 기준에서 라벨 불일치 가능성이 높았다는 수준이며 보안 전문가 2인 독립 감사(T8)는 수행하지 않았다. 다섯째, 비용 기반 지연 유발 공격(slowdown attack)[8] 노출을 DL-A 위에서 고정 페이로드 변형으로 실측한 결과(E15), 모델 접근 없이 공격 페이로드에 URL 인코딩 1회(`url_encode`)만 적용해도 1차(char-CNN)의 공격 넘김률 r_A가 B0 기준 0.27% → 8.76%로 약 **32**배, 공격 요청당 기대 지연이 0.73ms → 4.64ms로 **6.4**배 늘었다 — 즉 DeepSloth(ICLR'21)[8]의 slowdown 공격이 모델 접근 없는 problem-space 페이로드 변형만으로 딥러닝 캐스케이드 웹 공격 탐지에 이전되며, 2차가 1차보다 약 76배 무거워 넘김 증가가 비용으로 크게 증폭된다(판정 4개 변형 평균 r_A 배율은 B0 ×11.0, G2 ×2.78로 G2/B0 = 0.25여서 클래스별 τ의 대가 조항은

미발동이나, E13에서 G2의 비용 이득이 재현되지 않았으므로 이를 안전 수단으로 확장할 수는 없다). 다만 변형된 공격 요청도 기대 지연 4.64ms로 DistilBERT 단독(46.1ms)보다는 약 10배 싸서 공격 비율 3.3% 배포에서 평균 비용 증가는 약 0.13ms에 그친다. 오히려 같은 문단에서 지적해야 할 더 심각한 한계는 url_encode 시 **1**차의 공격 판정률이 **99.4% → 86.7%**로 떨어졌다는 점이다. 1차가 정상으로 확신한 변형 공격은 2차로 넘어가지 않으므로 그대로 놓치게 되어, 비용 증가가 아니라 탐지 회피**(evasion)** 문제가 발생한다(적응형 공격 및 2차까지 포함한 변형 최종 정확도는 폴드별 DistilBERT 가중치를 저장하지 않아 측정하지 않았다). 여섯째, 평가는 단일 데이터셋(SR-BH 2020, 정상 요청의 대부분이 스캐너 탐침)에서 수행되었다.

VII. 결론

본 논문은 인라인 WAF 환경에서 경량 문자 수준 CNN(char-CNN)과 사전학습 트랜스포머(DistilBERT)를 결합한 딥러닝 캐스케이드 및 조기 종료 구조의 속도–정확도 교환과 넘김 판단의 한계를 실트래픽 기반 SR-BH 2020 데이터셋에서 사전 등록 기준에 따라 평가하였다. 실험 결과, char-CNN 내부의 커널 3 분기에 보조 헤드를 결합 학습한 조기 종료 구조(DL-C)는 두 seed 모두 단독 모델과 동등한 정확도를 **3.5~3.6**배 빠르게 냈으나 그 원천은 넘김 판단이 아니라 결합 학습된 보조 헤드 자체의 모델 압축 효과였다. 한편 char-CNN → DistilBERT 캐스케이드(DL-A)는 시험 요청의 0.7~0.8%만 넘기면서 DistilBERT의 핵심 기여인 SQLi→CmdI 오탐 감소분의 절반 이상(**56~74%**)을 물려받고 실사용자 정상 기준 합산 **10.6~12.3**배 가속(결합 학습 k3 출구를 1차로 결합한 E14는 **23.8~29.3**배)을 달성하였으며, 원 논문 설정(128 토큰)의 문맥 잘림 손실(−1.70~−1.96%p)도 규명하였다. 그러나 DL-A의 사전 정확도 허용 기준(손실 0.30%p 이내)은 기본 seed 42(**−0.03%p**)에서만 충족하고 확인용 seed 43(**−0.32%p**)에서는 미세조정 편차로 재현되지 않았으며, 단일 임계값의 실사용자 정상 과잉 넘김(r_N = 33.6%)을 보완하려던 예측 클래스별 임계값(G2) 또한 새 seed 44·45(E13)에서 재현되지 않았고, URL 인코딩 변형에 의한 공격 지연 증폭(**6.4**배) 및 1차 탐지 회피 노출(E15)이 확인되었다. 결국 웹 공격 탐지 캐스케이드에서 핵심 병목은 넘김 임계값 조정이 아니라 1·2차 지연 격차와 1차 모델 자체의 판별력 및 변형 강건성에 있으며, 캐스케이드의 성능 주장은 반드시 종단간 공정 지연, 배포 공격 기저율, 실사용자 정상 넘김률, 다중 seed 분산을 함께 보고할 때에만 성립한다.

References

[1] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," Proceedings of the 31st USENIX Security Symposium, pp. 3971-3988, Aug. 2022.

[2] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," Advances in Neural Information Processing Systems (NIPS), pp. 649-657, Dec. 2015.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, Oct. 2019.

[4] X. Wang, D. Kondratyuk, E. Christiansen, K.M. Kitani, Y. Movshovitz-Attias, and E. Eban, "Wisdom of committees: an overlooked approach to faster and more accurate models," International Conference on Learning Representations (ICLR), Apr. 2022.

[5] S. Teerapittayanon, B. McDanel, and H.T. Kung, "BranchyNet: fast inference via early exiting from deep neural networks," Proceedings of the 23rd International Conference on Pattern Recognition (ICPR), pp. 2464-2469, Dec. 2016.

[6] K. Tasdemir, R. Khan, F. Siddiqui, S. Sezer, F. Kurugollu, S.B. Yengec-Tasdemir, and A. Bolat, "Advancing SQL injection detection for high-speed data centers: a novel approach using cascaded NLP," arXiv:2312.13041, Dec. 2023.

[7] W. Jitkrittum, N. Gupta, A.K. Menon, H. Narasimhan, A.S. Rawat, and S. Kumar, "When does confidence-based cascade deferral suffice?," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2023.

[8] S. Hong, Y. Kaya, I.-V. Modoranu, and T. Dumitraş, "A panda? No, it's a sloth: slowdown attacks on adaptive multi-exit neural network inference," International Conference on Learning Representations (ICLR), May 2021.

[9] F. Sureda Riera, J. Bermejo Higuera, J.R. Bermejo Higuera, J.J. Martínez Herraiz, and J.A. Sicilia Montalvo, "A new multi-label dataset for web attacks CAPEC classification using machine learning techniques," Computers & Security, 120, 102788, Sep. 2022.

[10] H. Osama, O. Elebiary, Y. Qassim, M. Amgad, A. Maghawry, A. Saafan, and H. Ghalwash, "Enhanced web payload classification using WAMM: an AI-based framework for dataset refinement and model evaluation," arXiv:2512.23610, Dec. 2025.

[11] J.R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," Scientific Reports, 14, 1803, Jan. 2024.

[12] K. Luo, Y. Xie, S. Zhao, and J. Pan, "WADBERT: dual-channel web attack detection based on BERT models," arXiv:2601.21893, Jan. 2026.

[13] M. Bitussi and R. Doriguzzi-Corin, "X-WAD: explainable web anomaly detection," arXiv:2608.27172, Aug. 2026.

[14] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," International Conference on Learning Representations (ICLR), Apr. 2017.

[15] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," Advances in Neural Information Processing Systems (NIPS), pp. 4878-4887, Dec. 2017.

[16] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 7076-7087, Jul. 2020.

[17] S. Enomoto and T. Eda, "Learning to cascade: confidence calibration for improving the accuracy and computational cost of cascade inference systems," Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), pp. 7331-7339, Feb. 2021.

[18] C. Guo, G. Pleiss, Y. Sun, and K.Q. Weinberger, "On calibration of modern neural networks," Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321-1330, Aug. 2017.

[19] V.J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J.S. Gardner, I. Hubara, S. Idgunji, T.B. Jablin, J. Jiao, T.S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A.T.R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "MLPerf inference benchmark," Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA), pp. 446-459, May 2020.

[20] S. Axelsson, "The base-rate fallacy and its implications for the difficulty of intrusion detection," Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS), pp. 1-7, Nov. 1999.

[21] C. Nadeau and Y. Bengio, "Inference for the generalization error," Machine Learning, 52(3), pp. 239-281, Sep. 2003.